

**LAPORAN TUGAS BESAR UAS
TUGAS PROYEK AKHIR TEKNIK KOMPILASI INI AKAN
MENGIMPLEMENTASIKAN KONSTRUKSI PENANGANAN
ERROR (ERROR HANDLER: TRY-CATCH) MENGGUNAKAN
BAHASA PEMROGRAMAN PYTHON**



TEKNIK KOMPILASI

Disusun Oleh:

SOMANTRI - 221011401711

**FAKULTAS ILMU KOMPUTER
PROGRAM STUDI TEKNIK INFORMATIKA S1
UNIVERSITAS PAMULANG
2025/2026**

Laporan Tugas Proyek Akhir: Representasi Tahapan Kompilasi (Try-Catch Error Handler)

1. Pilihan Konstruksi

Konstruksi sintaksis yang dipilih untuk diimplementasikan adalah: **Penanganan Error (Error Handler: try-catch)**. Konstruksi ini mendeteksi blok kode yang berpotensi menghasilkan *exception* (try) dan mengalihkan alur eksekusi ke blok penanganan jika terjadi kesalahan (catch).

2. Pattern (Pola Sintaks)

Pola tata bahasa untuk konstruksi try-catch ini didefinisikan menggunakan pendekatan **Backus-Naur Form (BNF)** sederhana sebagai berikut:

BNF

```
<try_catch_stmt> ::= "try" "{" <statements> "}" "catch" "(" <exception_type> <identifier> ")"  
"{" <statements> "}"
```

```
<exception_type> ::= "Exception" | "TypeError" | "ValueError"
```

```
<statements> ::= <identifier> "=" <expression>
```

```
<expression> ::= <identifier> <operator> <value> | <value>
```

```
<operator> ::= "+" | "-" | "*" | "/"
```

```
<identifier> ::= [a-zA-Z][a-zA-Z0-9]*
```

```
<value> ::= [0-9]+
```

3. Implementasi Program (Python)

Program di bawah ini menyimulasikan 4 tahapan kompilasi mendasar: **Analisis Leksikal**, **Analisis Sintaksis (AST)**, **Analisis Semantik (Pengecekan Tipe)**, dan **Generasi Kode Antara (TAC)**.

Python

```
import re
```

```
class TryCatchCompiler:
```

```
    def __init__(self, source_code):
```

```
        self.source_code = source_code
```

```
        self.label_counter = 1
```

```
    def new_label(self, prefix="L"):
```

```
        lbl = f"{prefix}{self.label_counter}"
```

```
self.label_counter += 1
return lbl
```

1. ANALISIS LEKSIKAL (Lexical Analysis)

```
def lexical_analysis(self):
```

Memisahkan token berdasarkan simbol-simbol kunci pemrograman

$$\text{delimiters} = \mathbf{r}'(\setminus \{\setminus\} \setminus (\setminus))'$$

Pisahkan token dan bersihkan whitespace yang tidak perlu

```
raw_tokens = re.split(delimiters, self.source_code)
```

```
tokens = []
```

```
for t in raw_tokens:
```

```
t = t.strip()
```

if t:

```
# Jika token bukan simbol tunggal, pecah berdasarkan spasi (untuk keyword/variabel)
```

```
if t in ['{', '}', '(', ')']:
```

```
tokens.append(t)
```

else:

```
tokens.extend(t.split())
```

return tokens

2. ANALISIS SINTAKSIS & SEMANTIK (Syntax & Semantic Analysis)

```
def parse_and_analyze(self, tokens):
```

Validasi struktur dasar try-catch via simulasi AST parsing

if 'try' not in tokens or 'catch' not in tokens:

```
raise SyntaxError("Sintaks Error: Struktur 'try' atau 'catch' tidak ditemukan.")
```

try:

Ekstraksi isi blok Try

```
try_idx = tokens.index('try')
```

```
try start = tokens.index('{', try_idx)
```

```

try_end = tokens.index('}', try_start)
try_body = " ".join(tokens[try_start + 1 : try_end])

# Ekstraksi parameter Exception pada Catch
catch_idx = tokens.index('catch')
param_start = tokens.index('(', catch_idx)
param_end = tokens.index(')', param_start)
catch_params = tokens[param_start + 1 : param_end]

if len(catch_params) < 2:
    raise SyntaxError("Sintaks Error: Parameter catch harus berisi (TipeException Variabel).")

exception_type = catch_params[0]
exception_var = catch_params[1]

# --- ANALISIS SEMANTIK (Validasi Tipe Data / Aturan) ---
valid_exceptions = ["Exception", "TypeError", "ValueError", "ArithmeticException"]
if exception_type not in valid_exceptions:
    raise TypeError(f"Semantik Error: Tipe Exception '{exception_type}' tidak dikenali.")

# Ekstraksi isi blok Catch
catch_body_start = tokens.index('{', param_end)
catch_body_end = tokens.index('}', catch_body_start)
catch_body = " ".join(tokens[catch_body_start + 1 : catch_body_end])

return try_body, exception_type, exception_var, catch_body

except ValueError:

```

```
        raise SyntaxError("Sintaks Error: Tanda kurung '{ }' atau '( )' tidak cocok/tidak lengkap.")
```

3. GENERASI KODE ANTARA / THREE-ADDRESS CODE (TAC)

```
def generate_tac(self):
    tokens = self.lexical_analysis()
    try_body, exc_type, exc_var, catch_body = self.parse_and_analyze(tokens)

    label_catch = self.new_label("CATCH")
    label_end = self.new_label("END")

    tac = []
    tac.append(";; --- BEGIN TRY-CATCH BLOCK ---")
    tac.append(f"setup_error_handler {label_catch} on {exc_type}")
    tac.append(f"// Blok Try:")
    tac.append(f"    {try_body}")
    tac.append(f"pop_error_handler")
    tac.append(f"goto {label_end}")

    tac.append(f"{label_catch}:")
    tac.append(f"// Blok Catch (Menerima {exc_var}):")
    tac.append(f"    {catch_body}")

    tac.append(f"{label_end}:")
    tac.append(";; --- END TRY-CATCH BLOCK ---")

    return "\n".join(tac)

# --- UJI COBA SIMULASI KOMPILOTOR ---

if __name__ == "__main__":
    # Contoh Input Kode Sumber yang akan dikompilasi
```

```
source_code = "try { result = x / y } catch ( ArithmeticException e ) { result = 0 }"
```

```
compiler = TryCatchCompiler(source_code)
```

```
print("=====")
```

```
print(" KODE SUMBER INPUT:")
```

```
print(f" {source_code}")
```

```
print("=====")
```

```
# 1. Tampilkan Hasil Analisis Leksikal
```

```
print("\n1. --- Hasil Analisis Leksikal (Tokens) ---")
```

```
tokens = compiler.lexical_analysis()
```

```
print(tokens)
```

```
# 2 & 3. Tampilkan Hasil Parsing Validasi Sintaksis & Semantik
```

```
print("\n2 & 3. --- Analisis Sintaksis & Semantik ---")
```

```
try:
```

```
    try_b, exc_t, exc_v, catch_b = compiler.parse_and_analyze(tokens)
```

```
    print("[✓] Sintaks Valid!")
```

```
    print(f"[✓] Semantik Valid! Tipe Exception '{exc_t}' terdaftar.")
```

```
    print(f"    - Isi Blok Try   : {try_b}")
```

```
    print(f"    - Parameter Catch: {exc_v} ({exc_t})")
```

```
    print(f"    - Isi Blok Catch : {catch_b}")
```

```
except (SyntaxError, TypeError) as error:
```

```
    print(f"[✗] Error Terdeteksi: {error}")
```

```
# 4. Tampilkan Hasil Generasi TAC
```

```
print("\n4. --- Generasi Three-Address Code (TAC) ---")
```

```
try:
```

```
    print(compiler.generate_tac())
```

except Exception as e:

```
print(f"Gagal generate TAC karena error: {e}")
```

4. Penjelasan Tahapan Kompilasi Proyek

Berdasarkan program simulator di atas, berikut rincian proses kompilasi kode try-catch dari input hingga terbentuknya kode antara:

1. **Analisis Leksikal (Lexer/Scanner):** Membaca string kode sumber karakter demi karakter dan mengelompokkannya ke dalam satuan kata terkecil yang valid bernama **Token**.
 - *Input:* "try { result = x / y } catch (ArithmeticException e) { result = 0 }"
 - *Output:* ['try', '{', 'result', '=', 'x', '/', 'y', '}', 'catch', '(', 'ArithmeticException', 'e', ')', '{', 'result', '=', '0', '}']
2. **Analisis Sintaksis (Parser):** Memeriksa apakah susunan token sudah sesuai dengan aturan hukum tata bahasa (BNF). Di sini sistem memastikan struktur try { ... } catch (...) { ... } ditulis lengkap dengan tanda kurung yang seimbang. Jika kurung kurawal tidak lengkap, compiler akan melemparkan **SyntaxError**.
3. **Analisis Semantik (Semantic Analyzer):** Memeriksa makna/logika dari kode. Pada simulator ini, analisis semantik melakukan **validasi tipe exception**. Program memeriksa apakah **ArithmeticException** terdaftar dalam sistem compiler internal. Jika user menulis nama exception sembarangan (misal: catch (SalahKetikException e)), sistem akan memblokir proses kompilasi dengan pesan **TypeError: Semantik Error**.
4. **Generasi Kode Antara / Three-Address Code (TAC):** Mentransformasikan struktur sintaksis tinggi ke bentuk instruksi linear tingkat rendah yang menggunakan *jump* (kondisional/unconditional) dan penanda label.
 - `setup_error_handler CATCH1 on ArithmeticException` bertindak mendaftarkan alamat *offset* penanganan jika operasi di dalam blok try gagal.
 - `pop_error_handler` bertindak menghapus pengawasan *error* jika blok try sukses berjalan tanpa kendala, lalu melompat langsung melewati blok catch (goto END1).